Song popularity prediction

```
library(tidyverse)
library(GGally)
library(naniar)
library(patchwork)
library(mice)
library(car)
library(e1071)
library(future)
library(doFuture)
library(finetune)
library(magrittr)
library(stacks)
library(DALEXtra)
library(tidymodels)

# defining a seed for the rest of the document
seed <- 202566346
```

Exploratory data analysis

We first import the dataset.

```
data <- read.csv("Song_reduced.csv")|>
  select(-binomial)
```

The data is made of 13161 rows and 15 columns (the name of the song, the popularity of the song and 13 audio features).

Descriptive summary statistics were first computed on the continuous numerical variables.

```
num.data <- data |> select(-c(song_name, Audio_mode, Key, Time_signature))

stats <- data.frame(
  Mean = round(sapply(num.data, mean, na.rm = TRUE),2),
  `Standard deviation` = round(sapply(num.data, sd, na.rm = TRUE),2),
  Minimum = round(sapply(num.data, min, na.rm = TRUE),2),
  `First quartile` = round(sapply(num.data, quantile, 0.25, na.rm = TRUE),2),
  Median = round(sapply(num.data, median, na.rm = TRUE),2),
  `Third quartile` = round(sapply(num.data, quantile, 0.75, na.rm = TRUE),2),
  Maximum = round(sapply(num.data, max, na.rm = TRUE),2)
)

knitr::kable(stats)
```

	Mean	Standard.deviation	Minimum	First.quartile	Median	Third.quartile	Maximum
song_popularity	52.94	21.87	0.00	40.00	55.00	69.00	100.00
Duration_ms	218184.07	61407.60	-805746.00	184868.00	211493.00	243487.00	1799346.00
Acousticness	0.26	0.31	-1.00	0.02	0.13	0.43	1.98
Danceability	0.63	0.16	0.00	0.53	0.64	0.75	0.98
Energy	0.64	0.21	0.00	0.51	0.67	0.81	1.00
Instrumentalness	0.08	0.22	0.00	0.00	0.00	0.00	1.00
Liveness	0.18	0.14	0.01	0.09	0.12	0.22	0.99
Loudness	-7.46	3.82	-36.28	-9.03	-6.57	-4.92	1.34
Speechiness	0.10	0.10	0.00	0.04	0.06	0.12	0.94
Tempo	120.99	29.67	-178.07	98.86	120.01	139.95	242.32
Audio_valence	0.53	0.25	0.00	0.33	0.53	0.72	0.98

The table above presents key summary statistics over all continuous numerical variables in the dataset (all variables except song name, audio mode, key and time signature).

The target variable in our analysis (song popularity) ranges from 0 to 100, with a mean around 53 and a median around 55. There are no values outside the range that was defined in the variable description, this variable does not require cleaning.

According to the presentation of the dataset, `Acousticness` has been defined to range between 0 and 1. However, as is presented in this table, the dataset has values for `Acousticness` outside of this range (the minimum is negative and the maximum is 1.98). This variable will have to be cleaned. The variable `Duration_ms` also has unlikely values : its minimum is negative, which is impossible for a song and its maximum is around 30 minutes (there might be 30 minutes songs but that seems unlikely, especially as this is much higher than the third quartile, this will have to be investigated further in the cleaning process). The feature `Tempo` will also require cleaning, as its minimum is negative and its maximum is around four beats per second, which is high. There are no other values that are out of the bounds that were defined in the dataset decsription.

The table above also shows that a lot of the numerical variables are skewed (as their medians are different from their means). In particular, `Acousticness`, `Instrumentalness`, `Liveness` and `Speechiness` are heavily left skewed. `Energy` and `Loudness` might also be slightly right-skewed.

Correlations between numerical variables were then examined. A full correlation matrix was first calculated, before the ten highest correlations (in absolute values) were retained.

```
cor_matrix <- cor(data |> select(-c(song_name, Key, Time_signature)), use = "pairwise.complete.obs")

highest_cor <- as_tibble(cor_matrix, rownames="Var1") |>
  pivot_longer(cols=-Var1, names_to="Var2", values_to="Correlation_coefficient") |>
  filter(Var1 != Var2) |>
  mutate(abs_corr=abs(Correlation_coefficient)) |>
  arrange(desc(abs_corr)) |>
  distinct(Correlation_coefficient, .keep_all=TRUE) |>
  slice_head(n=10) |>
  select(Var1, Var2, Correlation_coefficient)

knitr::kable(highest_cor)
```

Var1	Var2	Correlation_coefficient
Energy	Loudness	0.7597564
Acousticness	Energy	-0.6306144
Acousticness	Loudness	-0.5285863
Instrumentalness	Loudness	-0.4031876
Danceability	Audio_valence	0.3309183
Energy	Audio_valence	0.3189223
Energy	Instrumentalness	-0.2220679
Danceability	Speechiness	0.2059029
Loudness	Audio_valence	0.2024679
Instrumentalness	Audio_valence	-0.1863986

The table above shows the ten highest correlation coefficients in the dataset. `Energy`, `Loudness` and `Acousticness` are all very correlated. `Audio_valence` is also correlated with a lot of other predictors in the dataset, even though those correlation are not very strong. `song_popularity`, the target variable is not strongly correlated with any variable (the highest correlation between `song_popularity` and other variables is with `Instrumentalness`, at only -0.13)

Data cleaning and missingness analysis

Data cleaning

This section highlights the data cleaning methods that were used to clean the data: duplicated rows were first removed, categorical variables were transformed to factors, unusual values were analysed, before eventually being changed to NAs.

Duplicated rows

There are 853 duplicated rows in the original dataset. They were removed from the dataset in its clean version.

```
data.cleaned <- data |>
  distinct()
```

Unusual values

Unusual values were then dealt with.

As mentioned in the first question, `Acousticness`, `Tempo` and `Duration_ms` contain unusual values.

The values outside the 0 - 1 range in the `Acousticness` are impossible according to the definition of the variable. These values were therefore transformed to missing values.

`Duration_ms` also had unlikely high values (either really small values of really high values)

```
small_duration <- data.cleaned |>
  select(Duration_ms, song_name) |>
  arrange(Duration_ms) |>
  slice_head(n=10)
```

```
high_duration <- data.cleaned |>
  select(Duration_ms, song_name) |>
  arrange(desc(Duration_ms)) |>
  slice_head(n=10)

unlikely_duration <- bind_rows(high_duration, small_duration)

knitr::kable(unlikely_duration)
```

Duration_ms	song_name
1799346	Army Arrangement
1355938	Play
1047933	I Have a Dream - The Complete Speech of Martin Luther King Jr.
833493	Babarabatiri
829586	Mambo Inn
760973	Last Call
747222	1960 What?
736160	From Moment To Moment
710826	The End - Remastered
664573	Go Bang
-805746	Shakara (Oloje)
-318166	Can't Control It! - Extended Dub mix
-262826	Baby Got Back
-226906	Summertime Blues
-201413	Cleopatra
-181538	2 Tone Drip
12000	Gina Rodriguez - Latinos Trending Intro
26186	The Avengers
31373	Thank You
35920	Shirley Chisholm - 1972

The table above summarises the top and bottom ten songs for duration in the dataset. The unusually long songs are actually speeches or just very long songs after verification. Their duration were kept as is in the dataset. The negative values for duration are impossible and were therefore replaced by missing values. The unusually small values (the smallest positive values for duration are less than 30 seconds long) seem to actually be introduction songs, and were therefore not changed in the cleaned dataset.

`Tempo` also had unlikely high and low values.

```
small_tempo <- data.cleaned |>
  select(Tempo, song_name) |>
  arrange(Tempo) |>
  slice_head(n=15)

high_tempo <- data.cleaned |>
  select(Tempo, song_name) |>
  arrange(desc(Tempo)) |>
  slice_head(n=5)

unlikely_tempo <- bind_rows(high_tempo, small_tempo)

knitr::kable(unlikely_tempo)
```

Tempo	song_name
242.318	Thirteenth
214.686	Tiny Monsters
213.990	Permitame
212.058	Machika
211.357	Sword
-178.070	All Souled Out
-169.036	Yeah Right
-162.798	Fill in the Blank
-142.007	Racks on Me
-135.111	Tick Tick Boom
-125.999	The Way You Move (feat. Sleepy Brown) - Club Mix
-113.941	Honey All Over
-96.405	El Agua de Belén
-95.813	El Cantante
-62.978	To Stay
0.000	Aura
0.000	Gina Rodriguez - Latinos Trending Intro
46.591	. . . Familiar Place
51.607	Enter
52.181	Vasoonak

There are quite a lot of songs that have a negative or null tempo. These song tempos were replaced by missing values in the cleaned dataset. The song that has the highest tempo in the dataset has a tempo of more than 240 beats per minutes (meaning around 4 beats per seconds). That seems quite high, but not completely impossible, the highest values were kept as they were.

The repartition of the variable `Time_signature` also differs from the description that was given, as the values currently range from 0 to 5 when they were supposed to be between 3 and 7.

```
time_signature <- data.cleaned |>
  filter(Time_signature == 0 | Time_signature == 1) |>
  select(song_name, Time_signature)

knitr::kable(head(time_signature))
```

song_name	Time_signature
A Drop In The Ocean	1
All N My Grill (feat. Big Boi)	1
Aura	0
Be Somebody (feat. Kiiara) - Tyron Hapi Remix	1
Clockworks	1
Coração Vagabundo	1

The songs on the table above have a time signature marked as 0 or 1 in the dataset, but actually have a time signature of 4 (according to internet researches). It was therefore concluded that time signatures of 0 or 1 in the dataset were actually missing values.

The next chunk of code cleans the data as was highlighted before.

```
data.cleaned <- data.cleaned |>
  mutate(Acousticness=replace(Acousticness,
                              (Acousticness < 0) | (Acousticness > 1),
                              NA),
         Tempo=replace(Tempo,
                       Tempo <= 0,
                       NA),
         Duration_ms=replace(Duration_ms,
                             Duration_ms < 0,
                             NA),
         Time_signature=replace(Time_signature,
                                (Time_signature == 0) | (Time_signature == 1),
                                NA))
```

Categorical variables

The feature `Audio_mode` is a categorical variable. It was therefore transformed into a factor.

```
data.cleaned$Audio_mode <- as.factor(data.cleaned$Audio_mode)
```

The features `Key` and `Time_signature` are ordinal variables. They also have to be transformed into factors to be appropriately dealt with later.

```
data.cleaned$Key <- as.factor(data.cleaned$Key)
data.cleaned$Time_signature <- as.factor(data.cleaned$Time_signature)
```

Finally, the feature `song_name` was removed, as it was just useful to identify songs.

```
data.final <- data.cleaned |> select(-song_name)
```

Discussing missingness

Overview of the missingness

A missing variables summary was created to have an idea of the proportion of missing data in each column.

```
knitr::kable(miss_var_summary(data.final))
```

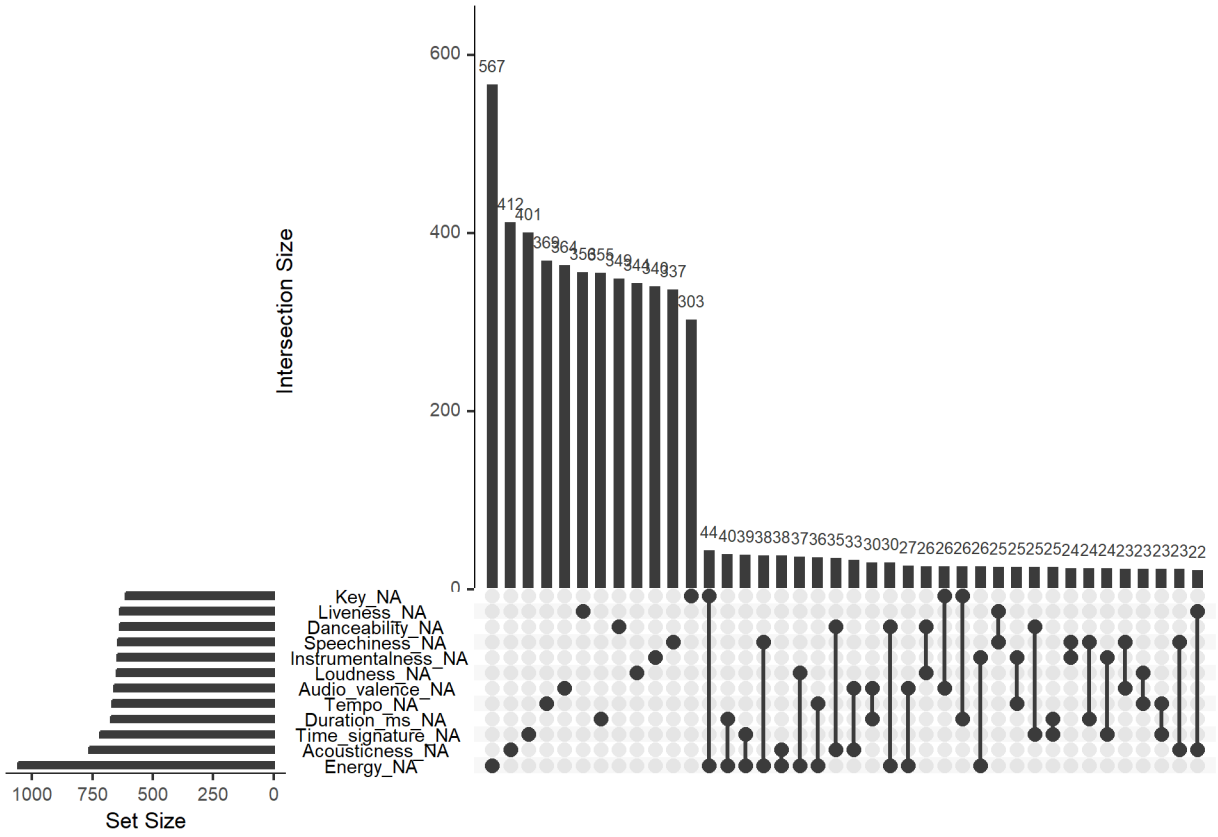
variable	n_miss	pct_miss
Energy	1055	8.57
Acousticness	763	6.20
Time_signature	718	5.83
Duration_ms	671	5.45
Tempo	667	5.42
Audio_valence	659	5.35
Loudness	648	5.26
Instrumentalness	645	5.24
Speechiness	642	5.22
Danceability	635	5.16

variable	n_miss	pct_miss
Liveness	634	5.15
Key	610	4.96
song_popularity	0	0
Audio_mode	0	0

The table above shows the amount of missing values in the dataset. `Energy` is the variable with the most missing values, with around 8.5% of missing values, followed by `Acousticness` that has 6.2% of missing values. `song_popularity` and `Audio_mode` are the only features with no missing data. All other features have around 5% of missing data (around 650 rows have missing data).

An upset plot was then created to understand the missingness in the dataset better.

```
gg_miss_upset(data.final, nsets=ncol(data.final)-2)
```



There are more than 5000 rows (out of 12308) that have missing values. Most rows having missing values only have one missing value, there are no rows with more than two missing values. It does not seem like there are any correlations between one column having missing values and other values missing (there is no particular combination of columns with a high number of missing values).

Type of missingness

The type of missingness in the dataset will now be discussed.

Little's test is first used to investigate whether the missing data is missing completely at random.

```
knitr::kable(mcar_test(data.final))
```

statistic	df	p.value	missing.patterns
4175.592	2914	0	261

The table above summarises the results of Little's test on our dataset. The p-value of the test is extremely close to 0, indicating strong evidence against MCAR. We therefore reject the null hypothesis that the missing data in the data is missing completely at random.

In general, it seems probable that the missingness in the dataset is related to the values of other audio features in the dataset.

- `Tempo`, `Key` and `Time_signature` : these features are objective musical measures. For these variables, the missingness can be related to `Speechiness` (if the track is not a song but a speech, it makes sense that there is no value for these features, so higher values of `Speechiness` may influence the missingness in these three features),
- `Duration_ms`, `Loudness` : these features are objective measures in a song. These features might be missing because of `song_popularity`. Unpopular songs that might be particularly old would be more likely to have these features missing, as they were not calculated, and, therefore, could not be extracted.
- `Energy`, `Acousticness`, `Audio_valence`, `Danceability`, `Liveness` and `Speechiness`, `Instrumentalness`: these features are not directly measured in the songs, they describe how a song feels to someone listening. For these variables, the missingness can be related to `song_popularity` (less popular songs are more likely to have missing values because they are less popular), `Speechiness` (if the speechiness value is high, indicating that the track might be a podcast or speech, it is more likely that values would be missing, as it is hard to evaluate the values of the other features in this category) or again `Liveness` (for songs performed live, that would therefore have higher values for liveness, it might be harder to calculate the values of the other features in this category)

It seems plausible that the missingness in the dataset is related to the values of other variables in the dataset. That would mean that the data missing is missing at random (MAR).

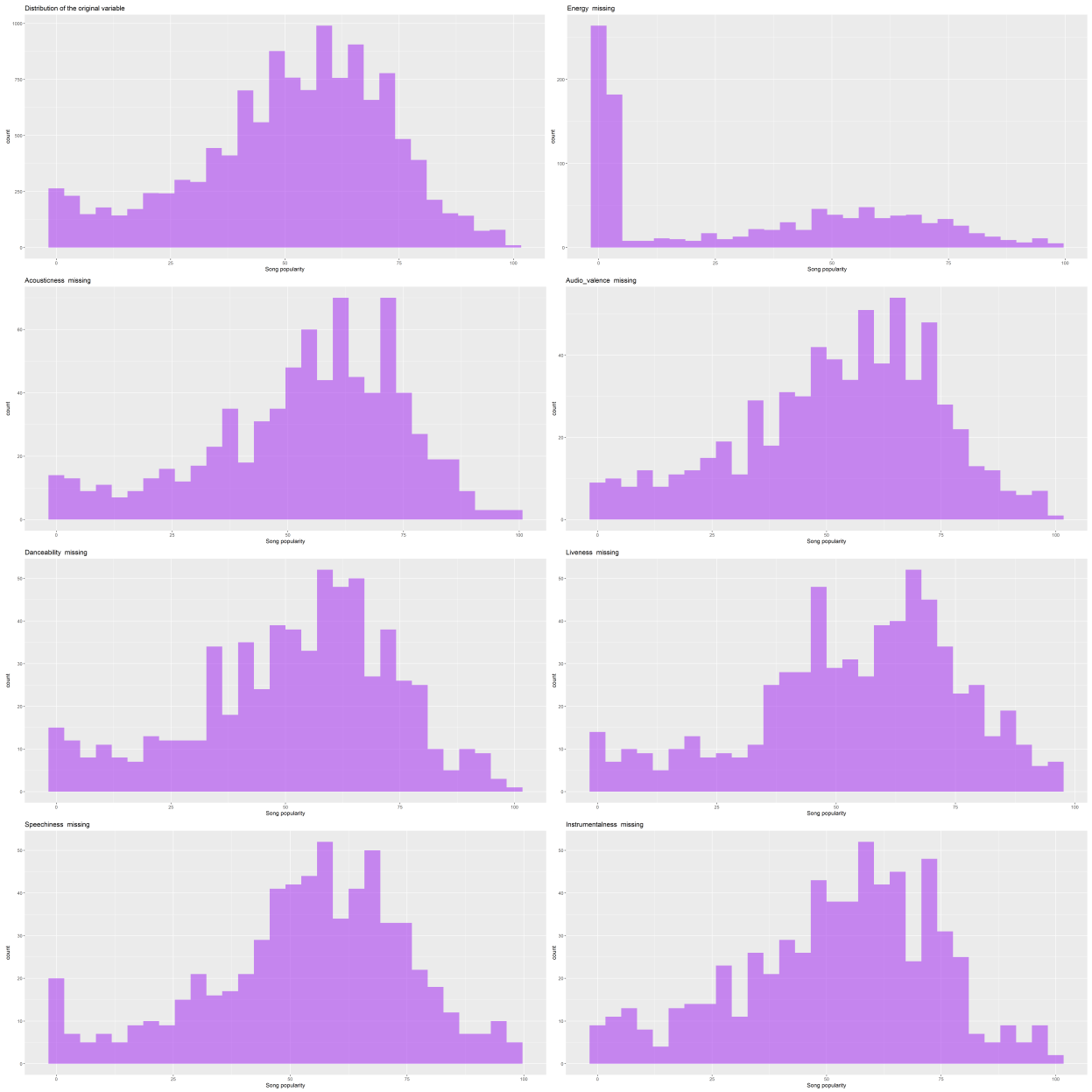
```
# original distribution of the variable
og_plot <- data.final |>
  ggplot() +
  geom_histogram(aes(x=song_popularity), alpha=0.5, fill="purple") +
  labs(title="Distribution of the original variable",
       x="Song popularity")

# function that plots the popularity for missing values
plot.missing.pop <- function(feature.name) {
  nrgy_plot <- data.final |>
  filter(is.na(.data[[feature.name]])) |>
  ggplot() +
  geom_histogram(aes(x=song_popularity), alpha=0.5, fill="purple") +
  labs(title=paste(feature.name, " missing"),
       x="Song popularity")
}

nrgy_plot <- plot.missing.pop("Energy")
acous_plot <- plot.missing.pop("Acousticness")
```

```
valence_plot <- plot.missing.pop("Audio_valence")
dance_plot <- plot.missing.pop("Danceability")
live_plot <- plot.missing.pop("Liveness")
speech_plot <- plot.missing.pop("Speechiness")
instru_plot <- plot.missing.pop("Instrumentalness")

(og_plot + nrgy_plot) / (acous_plot + valence_plot) / (dance_plot + live_plot) / (speech_plot + instru_
```



The plot above shows the difference in distribution between the original song popularity variable and the song popularities of songs having the variables of the last group missing. In general, it seems that the popularity when there is missing data is not that different from the original distribution, even though. The only flagrant effect is with missing values in **Energy**, where missing values seem to be linked to low song popularity. For the other variables in the group, it seems like tracks with missing values may be slightly less popular, the distributions seem to be more spread between 0 and 100 (the peak between 50 and 75 is

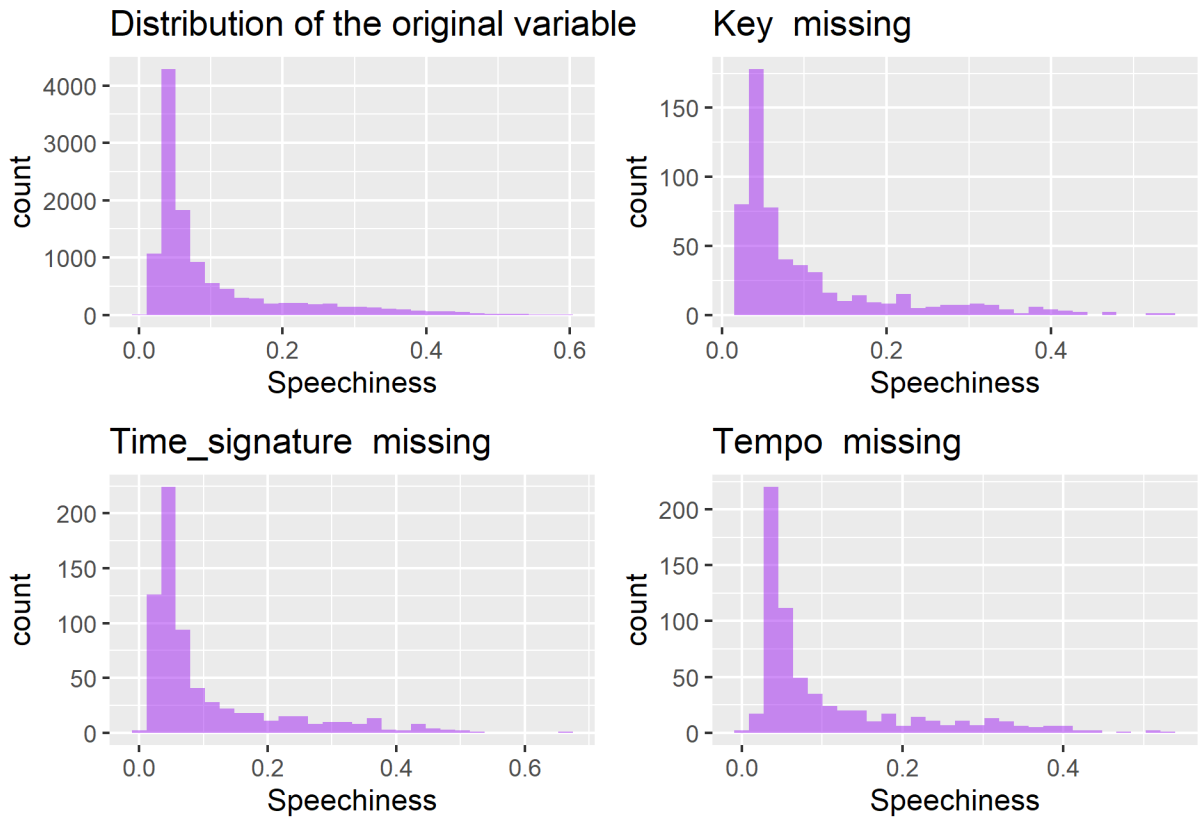
less pronounced, and there are more datapoints with a low song popularity) : this seems to be the case for `Acousticness`, `Audio_valence` and `Danceability`. `Speechiness` has a peak higher than normal for songs with a popularity close to 0.

```
# original distribution of the variable
og_plot <- data.final |>
  filter(Speechiness < 0.6) |> # removing unusually high values to have comparable plots
  ggplot() +
  geom_histogram(aes(x=Speechiness), alpha=0.5, fill="purple") +
  labs(title="Distribution of the original variable",
       x="Speechiness")

# function that plots the popularity for missing values
plot.missing.speech <- function(feature.name) {
  nrgy_plot <- data.final |>
  filter(is.na(.data[[feature.name]])) |>
  ggplot() +
  geom_histogram(aes(x=Speechiness), alpha=0.5, fill="purple") +
  labs(title=paste(feature.name, " missing"),
       x="Speechiness")
}

key_plot <- plot.missing.speech("Key")
signature_plot <- plot.missing.speech("Time_signature")
tempo_plot <- plot.missing.speech("Tempo")


(og_plot + key_plot) / (signature_plot + tempo_plot)
```



The plot above shows the difference in distribution between the original speechiness variable and the speechiness of songs having the variables of the first group missing. It seems like there is a slightly higher density for higher values of speechiness for distribution where variables are missing. However, again, this is not flagrant.

From these two plots, it seems like the missing data are missing at random, but that the missingness is not related to just one other variable, but to multiple other audio features present in the dataset, making it hard to visualise.

Imputing the missing data

As assessed previously, there is around 5% of missing data for most features that have missing data and it is plausible that the data missing is missing at random. Therefore, using multivariate imputation chained equations (MICE) is the best option. For continuous numerical variables (`Energy`, `Acousticness`, `Duration_ms`, `Tempo`, `Audio_valence`, `Loudness`, `Instrumentalness`, `Speechiness`, `Danceability` and `Liveness`), predictive mean matching was used to impute missing data. For the two other variables with missing values (`Key` and `Time_signature`), multinomial logistic regression was used, as they are categorical variables with more than one category. The algorithm was run for 15 iterations, in order to make sure convergence was reached and 15 datasets were imputed, in order to combine a high number of datasets to choose from at a lower computational cost.

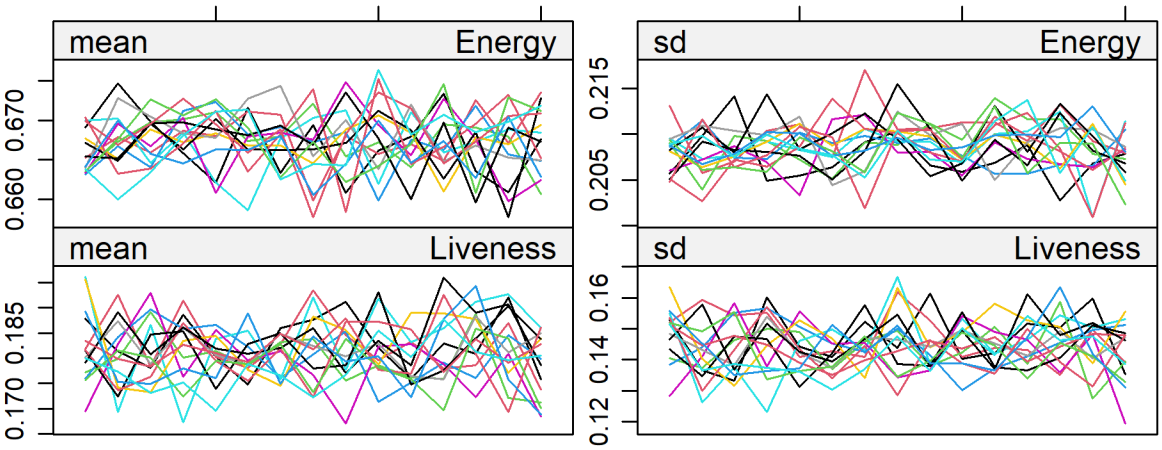
```
imp.mice <- mice(data.final,
                 m=15,
                 maxit=15,
                 printFlag=FALSE,
                 seed=seed)
```

Assessing the performance of the imputation

Convergence

Convergence was checked, using trace plots for numerical variables. The plots for Energy and Liveness are plotted here for reference, but all trace plots were investigated, and the same conclusions were reached on all plots.

```
plot(imp.mice,
     y=c("Energy", "Liveness"))
```



On these two plots, the mean and the standard deviation of the chosen variable in every imputed datasets seem to be stable across the iterations and the datasets. There is no trend in any trace plot, confirming that the model has converged.

Quality of the imputation

A linear regression model of `song_popularity` against all other variables in the dataset was fitted for each imputed dataset and the results were then pooled to assess the overall quality of imputation. The pooled r-squared was calculated to assess the goodness of fit of the linear regression across all datasets.

```
reg.models <- with(imp.mice,
                        lm(song_popularity~Energy+Acousticness+Duration_ms+Danceability+
                             Instrumentalness+Key+Loudness+Speechiness+
                             Tempo+Time_signature+Audio_valence+
```

```
                                Audio_mode+Liveness))

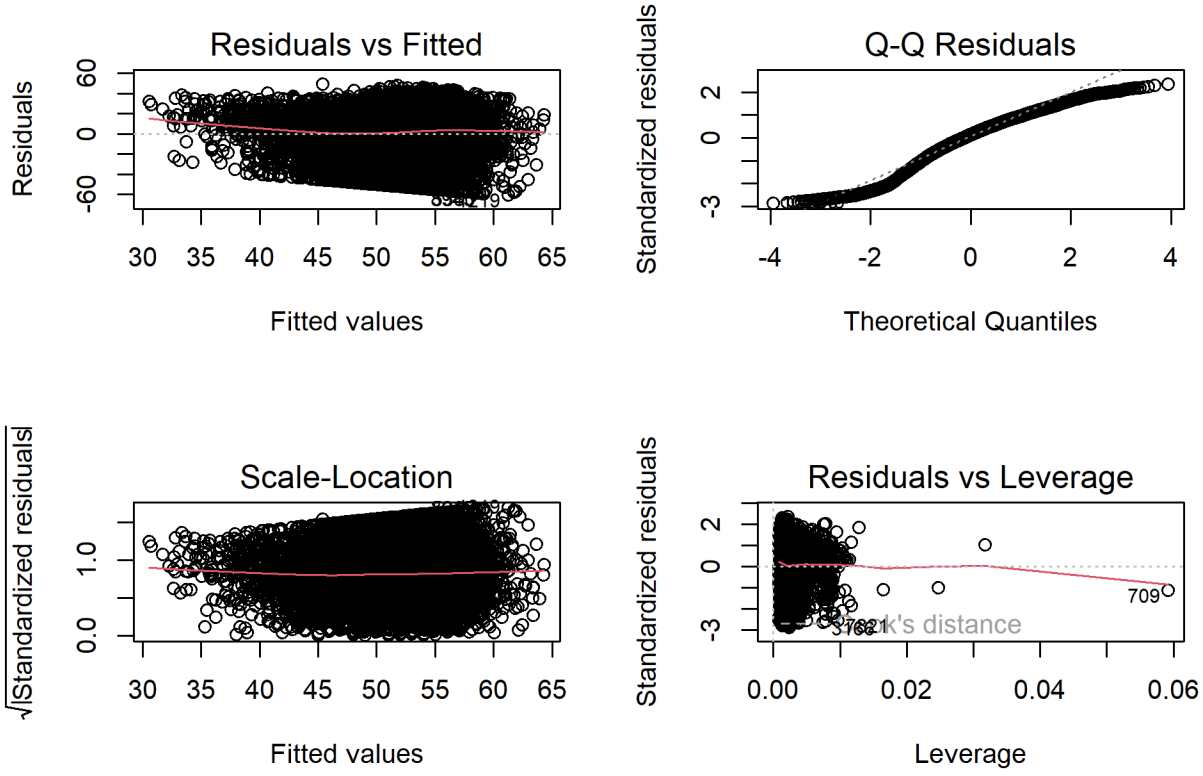
pool.r2 <- pool.r.squared(reg.models, adjusted=TRUE)
knitr::kable(pool.r2)
```

	est	lo 95	hi 95	fmi
adj R^2	0.0378423	0.0311922	0.0450798	0.0945732

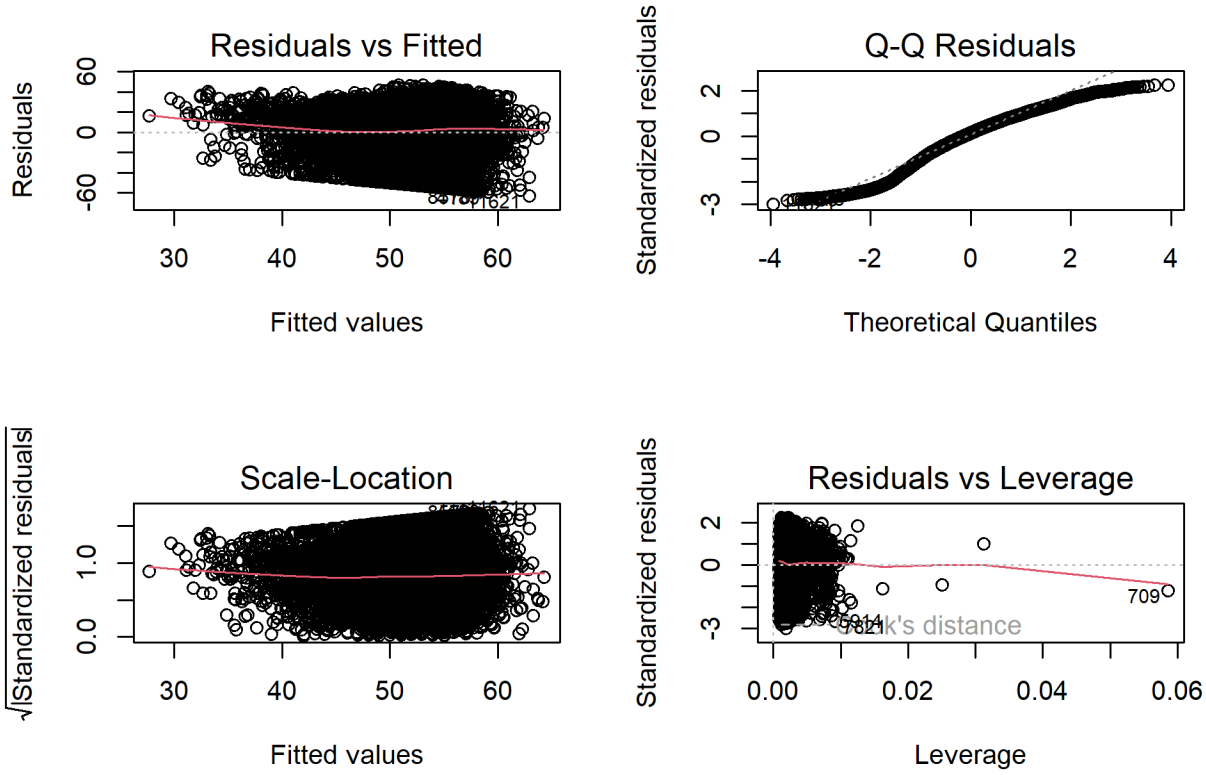
The pooled adjusted R-squared is 0.0378423, meaning that the linear regression model explains 3.78% of the variance in the target variable `song_popularity`. This is a very poor performance, but is equal to the adjusted R-squared obtained when running linear regression on the original data (minus rows containing missing data).

The assumptions of linear regression were then checked for two of the linear regression model that were fitted on the imputed datasets.

```
par(mfrow = c(2, 2))
plot(reg.models$analyses[[2]])
```




```
par(mfrow = c(2, 2))
plot(reg.models$analyses[[5]])
```



For all of these models, the residuals seem to be evenly distributed (as can be seen on the Residuals vs Fitted values plots), except maybe for low fitted values of popularity, that seem to have higher positive residuals. Both QQ-plots show that the residuals are slightly left-skewed, contradicting the normality assumption (even though there are a lot of data points, meaning that this should not be too concerning). The scale-location plots show no trend, confirming the homoscedasticity of the residuals. Overall, the assumptions of linear regression are checked for both of these models (there could be slight violation in the normality assumption though).

```
r2.perdataset <- sapply(reg.models$analyses, function(fit) {
  summary(fit)$r.squared
})
print(r2.perdataset)
```

```
##  [1] 0.03907301 0.03851742 0.03782654 0.04031440 0.04030242 0.03988482
##  [7] 0.03952702 0.04023531 0.03967156 0.03921864 0.04236603 0.03932861
## [13] 0.04087712 0.03932667 0.03940336
```

The eleventh imputed dataset has the highest r squared, this is the one that was chosen for further analysis.

```
data.complete <- complete(imp.mice, 11)
summary(data.complete)
```

```
##  song_popularity   Duration_ms       Acousticness        Danceability
##  Min.   :  0.00   Min.   :  12000   Min.   :0.0000014   Min.   :0.0000
##  1st Qu.: 40.00   1st Qu.: 184690   1st Qu.:0.0243750   1st Qu.:0.5300
```

```
##  Median : 54.00   Median : 211666   Median :0.1360000   Median :0.6430
##  Mean   : 51.65   Mean   : 218611   Mean   :0.2633591   Mean   :0.6301
##  3rd Qu.: 67.00   3rd Qu.: 244066   3rd Qu.:0.4390000   3rd Qu.:0.7450
##  Max.   :100.00   Max.   :1799346   Max.   :0.9960000   Max.   :0.9810
##
##      Energy        Instrumentalness          Key          Liveness
##  Min.   :0.00163   Min.   :0.0000000   0      :1462   Min.   :0.0109
##  1st Qu.:0.50400   1st Qu.:0.0000000   1      :1416   1st Qu.:0.0930
##  Median :0.67500   Median :0.0000133   7      :1331   Median :0.1220
##  Mean   :0.64446   Mean   :0.0840182   2      :1109   Mean   :0.1802
##  3rd Qu.:0.81600   3rd Qu.:0.0032900   9      :1098   3rd Qu.:0.2230
##  Max.   :0.99900   Max.   :0.9970000   11     :1038   Max.   :0.9860
##                                        (Other):4854
##     Loudness       Audio_mode  Speechiness         Tempo        Time_signature
##  Min.   :-36.281   0:4555     Min.   :0.0000   Min.   : 46.59   3:  527
##  1st Qu.: -9.113   1:7753     1st Qu.:0.0377   1st Qu.: 98.83   4:11619
##  Median : -6.624              Median :0.0549   Median :120.02   5:  162
##  Mean   : -7.520              Mean   :0.1004   Mean   :121.30
##  3rd Qu.: -4.939              3rd Qu.:0.1160   3rd Qu.:139.96
##  Max.   :  1.342              Max.   :0.9410   Max.   :242.32
##
##  Audio_valence
##  Min.   :0.0000
##  1st Qu.:0.3350
##  Median :0.5280
##  Mean   :0.5287
##  3rd Qu.:0.7280
##  Max.   :0.9840
##
```

The output above shows the summary statistics of all imputed variables, to make sure the imputation has been carried out correctly. There is no value out of the ranges that were described in the definitions of the dataset, it seems that the imputation has been done correctly.

Developping an ensemble model

```
plan(multisession, workers = 2)
registerDoFuture()
```

An ensemble model was developed to predict the popularity of a song based on audio features. 5 different models were integrated in the final ensemble model : linear regression, a support vector machine, an artificial neural network, a random forest and a gradient boosting model.

The data was split into a training and a testing dataset using stratified sampling. The training dataset was then partitioned in 5 different parts for cross-validation.

```
set.seed(seed)
song_split <- initial_split(data.complete, strata = song_popularity)
song_train <- training(song_split)
song_test  <-  testing(song_split)

song_folds <- vfold_cv(song_train, v = 5)
```

Feature preprocessing

The feature preprocessing reasoning is described here.

```
knitr::kable(table(data.complete$Time_signature))
```

Var1	Freq
3	527
4	11619
5	162

The feature `Time_signature` is categorical and is equal to 4 for around 95% of the rows in the dataset. As this feature is not informative for any model, it was removed from the list of predictors.

The skewness statistics are calculated on all numerical variables.

```
numerical_variables <- c("Energy", "Acousticness", "Audio_valence", "Duration_ms", "Danceability", "Liv

skew.df <- sapply(select(data.complete, numerical_variables),
                  skewness)

knitr::kable(skew.df)
```

	x
Energy	-0.6195219
Acousticness	1.0408598
Audio_valence	-0.0328183
Duration_ms	3.5171869
Danceability	-0.4105010
Liveness	2.2021037
Speechiness	2.2959358
Instrumentalness	2.8425676
song_popularity	-0.5195697

The distributions of the features `Audio_valence`, and `Danceability` are roughly symmetrical, no preprocessing was needed for these features.

`Energy`, `Liveness` and `Speechiness` are skewed : `Energy` is moderately left skewed while both `Liveness` and `Speechiness` are heavily right-skewed. Yeo-Johnson transformation will be applied to these variables to make them normal.

`Duration_ms` has the highest skewness statistic (4.58), indicating strong skewness. That is, however, due to just a few outliers that were investigated before. No preprocessing was therefore applied to this feature.

`Instrumentalness` has a high peak around 0 and another (much smaller one) around 1 : this variable was transformed into a categorical variable (0 : there are lyrics, 1 : the song is instrumental with a low threshold).

3 different recipes were used for the models.

- For SVM, neural network and XGBoost : SVMs and neural networks require normal numerical predictors normalized inputs and encoded categorical features. XGBoost only requires encoded categorical features, but as this was a recipe in which categorical features were properly encoded, it was also used

for XGBoost. The skewed features (`Energy`, `Acousticness`, `Liveness`, `Loudness` and `Speechiness`) were transformed using yeo-johnson transformation, in order for these features' distributions to be roughly normal. As discussed before, `Instrumentalness` was transformed into a categorical variable. Categorical variables (`Instrumentalness`, `Key` and `Time_signature`) were then one-hot encoded.

- For linear regression : a polynomal step was added to the normalized recipe described before
- For random forest : this model does not require normalization of numerical features or a specific handling of categorical features. No specific pre-processing has been used for this model.

```
normalized_rec <- rec_normalized <- recipe(song_popularity~Audio_valence+Duration_ms+Danceability+
                                           Energy+Liveness+Speechiness+Instrumentalness+
                                           Key+Audio_mode+Tempo+Loudness+Acousticness, data=song_trai
  step_YeoJohnson(Energy) |>
  step_YeoJohnson(Liveness) |>
  step_YeoJohnson(Speechiness) |>
  step_YeoJohnson(Acousticness) |>
  step_YeoJohnson(Loudness) |>
  step_cut(Instrumentalness, breaks=c(0, 0.5, 1))|>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_predictors())

poly_rec <- normalized_rec |>
  step_poly(Energy, Liveness, Speechiness, Audio_valence, Duration_ms, Danceability, Tempo, Acousticnes
  step_interact(~all_predictors():all_predictors())

noprepro_rec <- recipe(song_popularity~Audio_valence+Duration_ms+Danceability+
                                           Energy+Liveness+Speechiness+Instrumentalness+
                                           Key+Audio_mode+Tempo+Loudness+Acousticness,
              data=song_train)
```

Model specifications

Each model was then defined and specified.

For linear regression, the regularization parameters were tuned : the `penalty` parameter, that controls the total amount of regularization applied in the model and the `mixture` parameter, that controls the proportion of l1 (lasso) regularization in the model (with 1 being only l1 regularization and 0 being only l2 (ridge) regularization).

For the neural network model, the number of hidden units (in the single layer), the weight decay and the number of epochs were tuned.

For the gradient boosting algorithm, the parameters tuned were the following : the number of parameters randomly sampled at each split in the tree models, the minimum number of data points in a node for it to be split, the total number of trees in the model, the learning rate, the maximum depth of each tree in the algorithm, the minimum loss reduction required for a new node to be created and the depth of the trees in the models.

For the support vector machine, the cost parameter (that controls the cost of having a prediction in or on the wrong side of the margin) and the radial basis function parameter were tuned.

Finally for the random forest, the number of parameters randomly sampled at each split in the tree models, the minimum number of data points in a node for it to be split and the total number of trees in the model were tuned

```
linear_reg_spec <-
  linear_reg(penalty = tune(),
             mixture = tune()) |>
  set_engine('glmnet')

nnet_spec <-
  mlp(hidden_units = tune(),
      penalty = tune(),
      epochs = tune()) |>
  set_engine('nnet') |>
  set_mode('regression')

tree_spec <-
  boost_tree(mtry  = tune(),
             min_n = tune(),
             trees = tune(),
             learn_rate = tune(),
             loss_reduction=tune(),
             tree_depth=tune()) |>
  set_engine('xgboost') |>
  set_mode('regression')

svm_r_spec <-
  svm_rbf(cost = tune(),
          rbf_sigma = tune()) |>
  set_engine('kernlab') |>
  set_mode('regression')

rf_spec <-
  rand_forest(mtry = tune(),
              min_n = tune(),
              trees = tune()) |>
  set_engine('ranger') |>
  set_mode('regression')
```

The parameters ranges were then updated.

```
N_feature = ncol(song_train) - 1

# Specifying the neural network model parameter range
nnet_param <-
  nnet_spec |>
  extract_parameter_set_dials() |>
  update(hidden_units = hidden_units(c(10, 500)))

# Specifying the random forest model parameter range
rf_param <-
  rf_spec |>
  extract_parameter_set_dials() |>
  update(
    mtry  = mtry(c(1, N_feature)),
    trees = trees(c(500, 2000)),
    min_n = min_n(c(1, 100))
```

```
  )

# Specifying the XGBoost model parameter range
BRT_param <-
  tree_spec |>
  extract_parameter_set_dials() |>
  update(
    mtry  = mtry(c(1, N_feature)),
    trees = trees(c(500, 5000)),
    min_n = min_n(c(1, 100),),
    learn_rate = learn_rate(c(0.0001, 0.5))
  )

# Specifying the SVM model parameter range
svm_r_param <-
  svm_r_spec |>
  extract_parameter_set_dials() |>
  update(cost = cost(c(-3, 2)))
```

Each workflow sets were then created.

```
# Creating the workflow set for normalized models (SVM, neural network and linear regression)
normalized <-
  workflow_set(
    preproc = list(normalized = normalized_rec),
    models  = list(SVM_radial = svm_r_spec,
                   nnet       = nnet_spec,
                   BRT        = tree_spec)
  )

# Creating the workflow set for linear regression
poly <- workflow_set(
  preproc=list(poly = poly_rec),
  models=list(linear_reg = linear_reg_spec)
)

# Creating the workflow set for models that do not require pre-processing (random forest)
no_pre_proc <-
  workflow_set(
    preproc = list(simple = noprepro_rec),
    models = list(RF = rf_spec)
  )
```

The parameter ranges were updated in each workflow.

```
# Updating parameter range for neural network model
normalized %<>%
  option_add(param_info = nnet_param,
             id = 'normalized_nnet') |>
  option_add(param_info = BRT_param,
             id = 'normalized_BRT') |>
  option_add(param_info = svm_r_param,
             id = 'normalized_SVM_radial')
```

```
## # A workflow set/tibble: 3 x 4
##   wflow_id              info             option    result
##   <chr>                 <list>           <list>    <list>
## 1 normalized_SVM_radial <tibble [1 x 4]> <opts[1]> <list [0]>
## 2 normalized_nnet       <tibble [1 x 4]> <opts[1]> <list [0]>
## 3 normalized_BRT        <tibble [1 x 4]> <opts[1]> <list [0]>
```

```
# Updating parameter range for random forest and for XGBoost
no_pre_proc %<>%
  option_add(param_info = rf_param,
             id = 'simple_RF')
```

Finally, all three workflows were combined.

```
all_workflows <-
  bind_rows(no_pre_proc, normalized, poly) |>
  mutate(wflow_id = gsub('(simple_)|(normalized_)', '', wflow_id))
```

Parameter tuning

Racing grid search was then applied to the workflow to find the optimal hyperparameters for each model. For each model, 20 different combinations of hyperpaameters were tested (meaning that 20 different models were fitted).

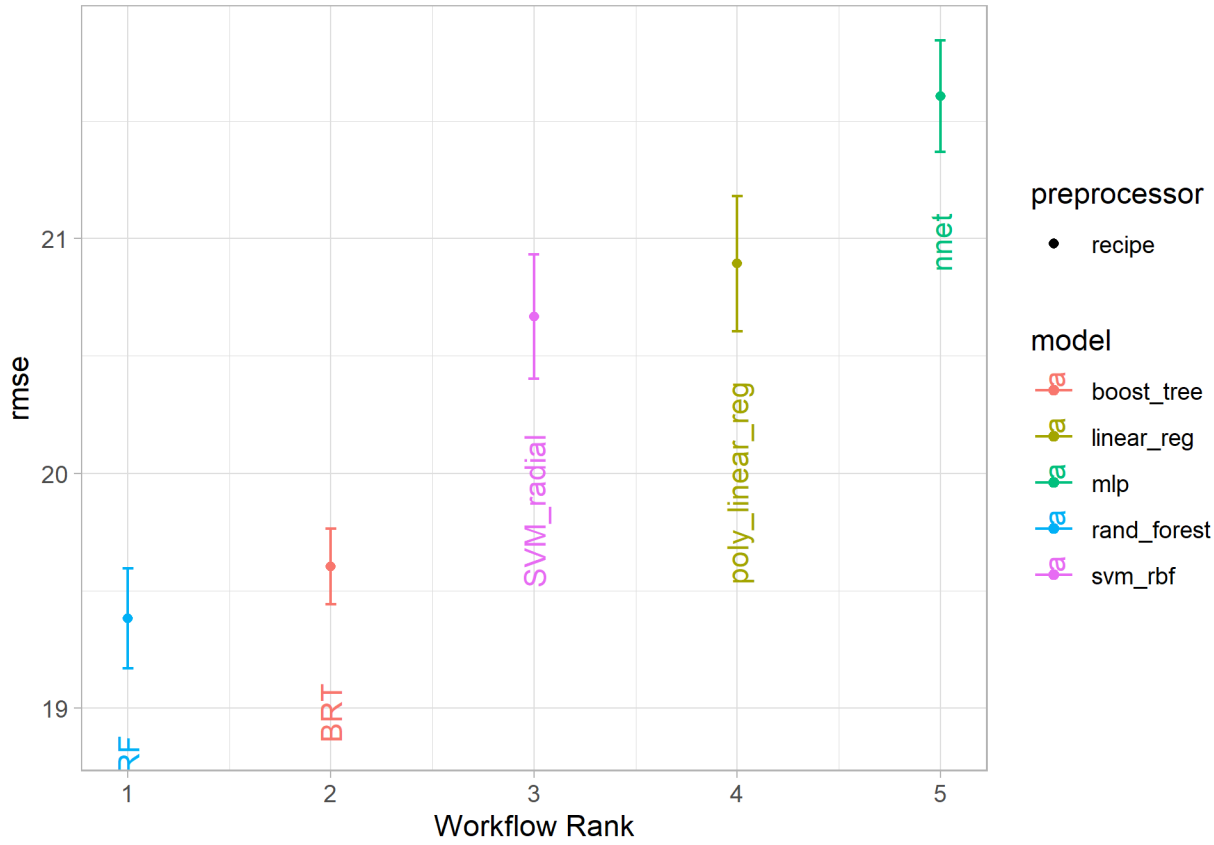
```
race_ctrl <-
  control_race(
    save_pred = TRUE,
    parallel_over = 'everything',
    save_workflow = TRUE
  )

race_results <-
  all_workflows %>%
  workflow_map(
      'tune_race_anova',
      seed      = seed,
      resamples = song_folds,
      grid      = 20,
      control   = race_ctrl,
      verbose   = TRUE
    )
```

Here is a comparison of the results for each model :

```
autoplot(
  race_results,
  rank_metric = 'rmse',
  metric = 'rmse',
  select_best = TRUE
) +
  geom_text(aes(y = mean - 1/2, label = wflow_id), angle = 90, hjust = 1) +
  #lims(y = c(0.065, 0.095)) +
```

```
  theme(legend.position = 'none') +
  theme_light()
```



For all models, the root mean squared error is around 20, which is quite bad, as it means the average distances between the predicted popularities and the actual values are close to 20 (which is a lot on the scale of 0 to 100 used in this dataset). The random forest performs the best, followed by gradient boosting: tree based ensemble models are the best models. This is coherent, as they are able to handle complex relationships between variables. The model that has the worst results is neural network, which is not surprising as the dataset is not very large compared to datasets on which neural network models are usually trained.

```
# for random forest
rf_results <-
  race_results %>%
  extract_workflow_set_result('RF') %>%
  select_best(metric = 'rmse')

RF_test_results <-
  race_results %>%
  extract_workflow('RF') %>%
  finalize_workflow(rf_results) %>%
  last_fit(split = song_split)

# gradient boosting
brt_results <-
  race_results %>%
  extract_workflow_set_result('BRT') %>%
```

```
  select_best(metric = 'rmse')

BRT_test_results <-
  race_results %>%
  extract_workflow('BRT') %>%
  finalize_workflow(brt_results) %>%
  last_fit(split = song_split)

# svm
svm_results <-
  race_results %>%
  extract_workflow_set_result('SVM_radial') %>%
  select_best(metric = 'rmse')

svm_test_results <-
  race_results %>%
  extract_workflow('SVM_radial') %>%
  finalize_workflow(svm_results) %>%
  last_fit(split = song_split)

# linear regression
lr_results <-
  race_results %>%
  extract_workflow_set_result('poly_linear_reg') %>%
  select_best(metric = 'rmse')

lr_test_results <-
  race_results %>%
  extract_workflow('poly_linear_reg') %>%
  finalize_workflow(lr_results) %>%
  last_fit(split = song_split)

# neural network
nnet_results <-
  race_results %>%
  extract_workflow_set_result('nnet') %>%
  select_best(metric = 'rmse')

nnet_test_results <-
  race_results %>%
  extract_workflow('nnet') %>%
  finalize_workflow(nnet_results) %>%
  last_fit(split = song_split)

brt_metrics <- collect_metrics(BRT_test_results)
rf_metrics <- collect_metrics(RF_test_results)
svm_metrics <- collect_metrics(svm_test_results)
lr_metrics <- collect_metrics(lr_test_results)
nnet_metrics <- collect_metrics(nnet_test_results)

model.comparison <- bind_rows(
  brt_metrics |> mutate(model="BRT"),
  rf_metrics |> mutate(model="RF"),
```

```
  svm_metrics |> mutate(model="SVM"),
  lr_metrics |> mutate(model="LR"),
  nnet_metrics |> mutate(model="NNET")
) |>
  pivot_wider(names_from=.metric, values_from=.estimate) |>
  select(model, rmse, rsq)

model.comparison
```

```
## # A tibble: 5 x 3
##   model  rmse    rsq
##   <chr> <dbl>  <dbl>
## 1 BRT    19.2 0.197
## 2 RF     19.1 0.204
## 3 SVM    20.5 0.105
## 4 LR     21.0 0.0496
## 5 NNET   21.5 0.0406
```

The table above summarises the RMSE and R squared of each of the best models. The best model is a random forest model with a root mean square error of 19.1 and a R squared of around 0.20. The next best model is a gradient boosting model, that has a slightly higher root mean square error (19.2). The next two best models are one SVM and one linear regression, with RMSE of 20.5 and 21.0. Finally, the worst model is the neural network model, with a RMSE of 21.5.

The best R squared value (random forest) is greater than 0.20, meaning that the best single models explain slightly more than 20% of the variation in `song_popularity`, while the worst models (neural network and linear regression) only explains around 4% of the variation in the target variable.

Ensemble model

A stacking model was then fitted. The candidates of the stacking model are the models that were previously tuned.

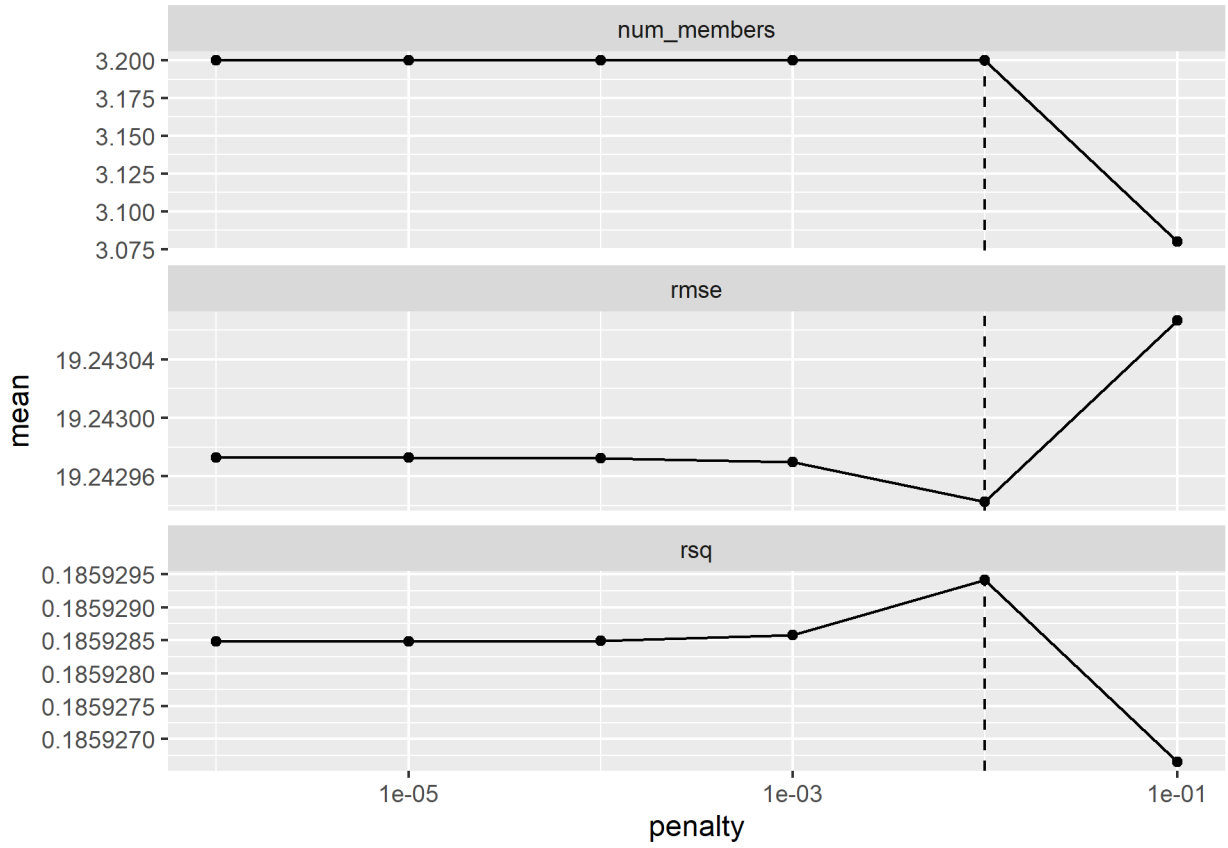
```
song_stack <-
  stacks() %>%
  add_candidates(race_results)
```

A regularized model was fitted, with values of l1 regularization ranging between 10^{-6} and 10^{-1}.

```
set.seed(seed)

ens <- blend_predictions(song_stack)

autoplot(ens)
```



The plot above shows the mean values of root mean squared error and r squared for each value of the regularization parameter that was tried. From this plot, he best values for this parameter is 0.01.

```
ens
```

```
## # A tibble: 3 x 3
##   member                       type        weight
##   <chr>                        <chr>        <dbl>
## 1 RFpre0_mod11_post0           rand_forest  0.740
## 2 BRTpre0_mod12_post0          boost_tree   0.285
## 3 SVM_radialpre0_mod13_post0 svm_rbf      0.183
```

The output above shows the models that were taken into account in the final ensemble model. The three best models that were described before were used in the ensemble model with the following weights : 0.74 for the best random forest model, 0.29 for the second best random forest model and 0.18 for the best gradient boosting model.

The ensemble model was finally fitted.

```
ens <- fit_members(ens)
```

```
reg_metrics <- metric_set(rmse, rsq)
ens_test_pred <-
  predict(ens, song_test) %>%
  bind_cols(song_test)
```

```
knitr::kable(ens_test_pred %>%
  reg_metrics(song_popularity, .pred))
```

.metric	.estimator	.estimate
rmse	standard	19.0096792
rsq	standard	0.2131191

The table above shows the metrics of the final fitted ensemble model. The root mean squared error on the test dataset is 19.0 (0.13 lower than the best single models, that had a RMSE of 19.13) and the r squared is 0.213 (which, again, is slightly better than the r squared for the single models). Roughly 21.3% of the variations in the popularity of a song are explained by the model, which is an increase of almost 1% compared to the best single model (random forest).

Output interpretation

To be able to interpret the model's output, several feature importance plots were produced based on the `explain_tidymodels` function.

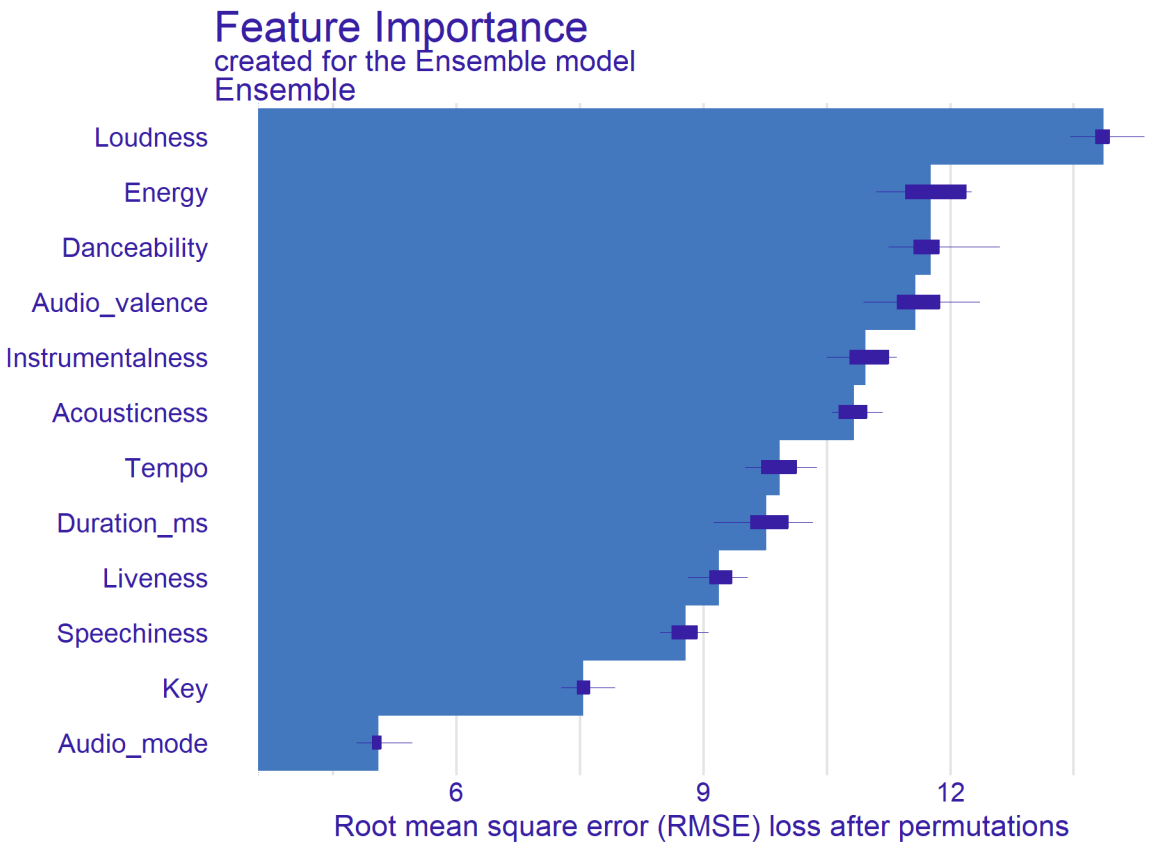
```
vip_features <- normalized_rec$var_info$variable

vip_train <-
  song_train %>%
  select(all_of(vip_features))

explainer_ens <-
  explain_tidymodels(
    ens,
    data = vip_train |> select(-song_popularity),
    y = song_train |> select(song_popularity),
    label = "Ensemble",
    verbose = FALSE
)
```

General feature importance was calculated at first. To get the importance of one feature, the observation for this feature are randomly shuffled multiple times, and the difference in root mean square error was calculated.

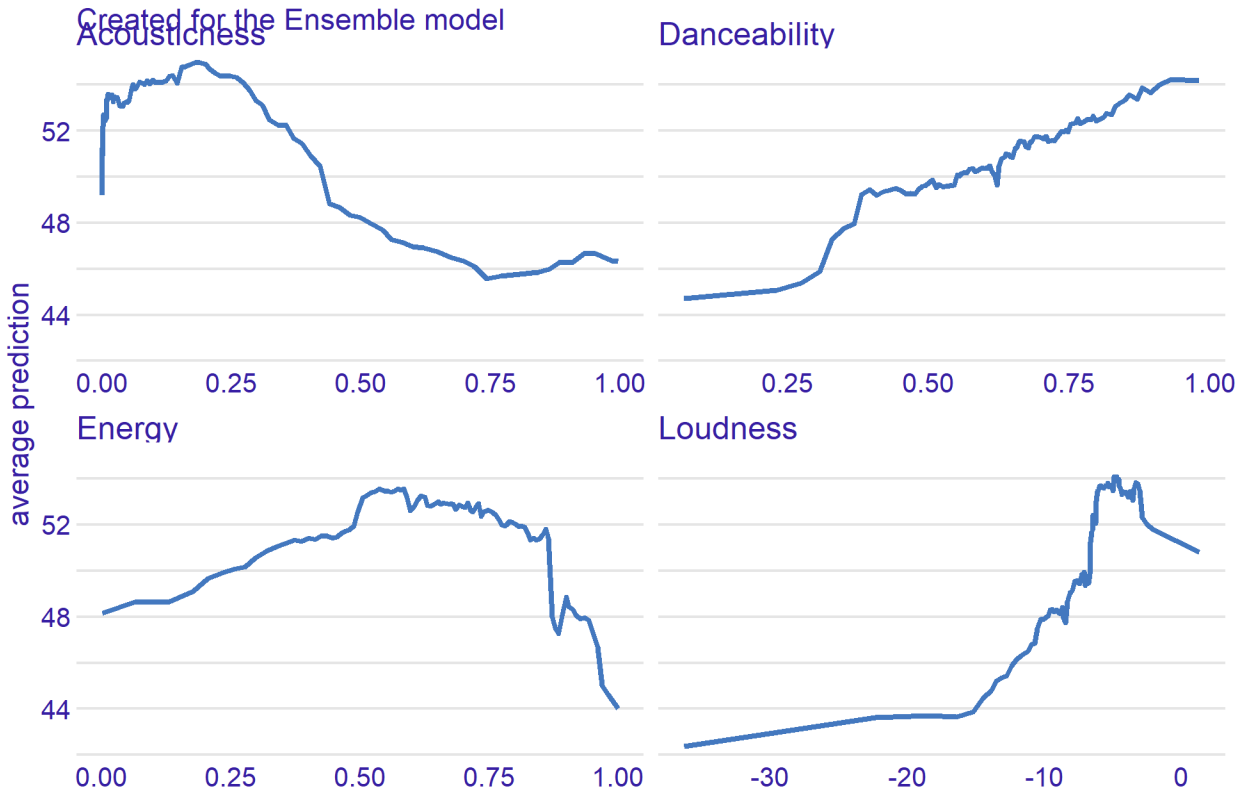
```
model_parts_ens <- model_parts(explainer_ens)
plot(model_parts_ens)
```



The plot above shows the results of the permutations method. `Loudness` is the most important feature in the model, with a RMSE loss around 14 after permutations. `Energy`, `Danceability`, `Audio_valence` `Instrumentalness` and `Acousticness` are the next most important features in the model. The least important features in the model are `Key` and `Audio_mode`.

Local dependence profiles were plotted for a few of the most important features in the dataset.

```
adp <- model_profile(
  explainer_ens,
  type = 'conditional',
  N = 100,
  variables = c("Loudness", "Energy", "Danceability", "Acousticness")
)
plot(adp)
```



Overall, acousticsness has a negative impact on the prediction of song popularity, higher acousticness means lower song popularity predictions. More precisely, the predictions are stable when acousticness is low (between 0 and 0.2), before decreasing suddenly for values of acousticness between 0.2 and 0.4. The average predicted song popularity is stable / slightly decreases for values of acousticness greater than 0.4.

As energy increases from 0 to around 0.75, the average predicted song popularity increases. When energy keeps increasing, the average predicted song popularity sinks down, to reach a predicted song popularity of 44. This means that Energy has a positive impact on song popularity between 0 and 0.75, and negatively impacts the precisions for values of energy higher than 0.75.

As loudness increases (and all other variables are fixed), the average predictions start by getting lower, before getting higher and reaching a peak just before 0. After this peak, the song popularity decreases. This sudden decrease was expected, it seems to match the decrease in song popularity for higher values of energy (as the two variables are highly correlated, it would be expected that their plots would look similar).

Danceability has a positive impact on the popularity of a song. It seems that higher danceability imply higher predictions from the model.

Overall, songs with high predicted popularity are loud and energetic songs, with values of loudness between -5 dB and 0 dB and values of Energy between 0.6 and 0.75. Popular songs are easy to dance to and not acoustic songs in general.